

Rewording inheriting constructors (core issue 1941 et al)

ISO/IEC JTC1 SC22 WG21
P0136R0
Revises: N4429
Richard Smith
richard@metafoo.co.uk
2015-09-25

Fixing:

- [Core Issue 1573](#): Inherited constructor characteristics
- [Core Issue 1645](#): Identical inheriting constructors via default arguments
- [Core Issue 1715](#): Access and inherited constructor templates
- [Core Issue 1736](#): Inheriting constructor templates in a local class
- [Core Issue 1903](#): What declarations are introduced by a non-member using-declaration?
- [Core Issue 1941](#): SFINAE and inherited constructor default arguments
- [Core Issue 1959](#): Inadvertently inherited copy constructor
- [Core Issue 1991](#): Inheriting constructors vs default arguments

Changes since N4429

- in 7.3.3p15, don't restrict the set of inherited members
- in 7.3.3p17, make explicit that no access check is performed in "using Base::Base;"
- in 7.3.3p18, make it clearer that the accessibility of the base class is not considered when using a constructor inherited from a base class
- in 12.6.3.p1, make it clearer that we don't default-initialize the chosen base class
- in 12.6.3.p2, give a more complex example showing the Y != X case
- in 12.6.3, directly initialize the class from which the constructor was originally inherited rather than proceeding recursively, and clarify that parameter initialization is sequenced before subobject initialization

Overview

There are a number of active core issues relating to inheriting constructors, with a single common theme: an inheriting constructor does not act like any other form of *using-declaration*. The reason for this is that all other *using-declarations* make some set of declarations visible to name lookup in another context, but an inheriting constructor declaration declares a new constructor (or set thereof) that merely delegates to the original.

After some discussion in CWG, it is believed that these issues can all be resolved by changing the meaning of an inheriting constructor declaration from declaring a set of new constructors, to making a set of base class constructors visible in a derived class *as if* they were derived class constructors. (When such a constructor is used, the additional derived class subobjects will also be implicitly constructed as if by a defaulted default constructor.) Put another way: **this proposal makes inheriting a constructor act just like inheriting any other base class member, to the extent possible.**

This change does affect the meaning and validity of some programs, but these changes improve the consistency and comprehensibility of C++. CWG wishes to treat this change as a DR against C++11.

Examples of behavior changes

- SFINAE on inheriting constructors now works reliably

```
struct A {
    template<typename T> A(T, typename enable_if<sizeof(T) == 4, int>::type = 0);
};
struct B : A {
    using A::A; // formerly declared
                // #1: B(T)
                // #2: B(T, typename enable_if<...>::type)
                // now makes A::A(T, typename enable_if<...>::type = 0)
                // visible as a constructor of B

    B(...);
};
B b(123ull); // now ok; used to ignore SFINAE condition and select inherited constructor #1
```

- Hiding works the same as for other member *using-declarations* in the presence of default arguments

```
struct A {
    A(int a, int b = 0);
    void f(int a, int b = 0);
};
struct B : A {
```

```

    B(int a);          using A::A;
    void f(int a); using A::f;
};
struct C : A {
    C(int a, int b = 0);          using A::A;
    void f(int a, int b = 0); using A::f;
};

B b(0); // was ok, now ambiguous
b.f(0); // ambiguous (unchanged)

C c(0); // was ambiguous, now ok
c.f(0); // ok (unchanged)

```

- Inheriting constructor parameters are no longer copied

```

struct A { A(const A&) = delete; A(int); };
struct B { B(A); void f(A); };
struct C : B { using B::B; using B::f; };
C c({0}); // was ill-formed, now ok (no copy made)
c.f({0}); // ok (unchanged)

```

- Ellipsis parameters used to be unimplementable on some ABIs

```

struct A { A(const char *, ...); };
struct B : A { using A::A; };
B b("%d %d", 1, 2); // ok, now implementable

```

- Access check now exactly matches the base class member

```

class A {
public:
    template<typename T> A(T, typename T::type);
private:
    A(int);
    friend void f();
};
class B {
    typedef int type;
    friend class A;
};
struct C : A {
    using A::A;
};

void f() {
    A a1(B(), 0); // ok (unchanged)
    C c1(B(), 0); // was ill-formed (implicit C constructor can't access B::type), now ok

    A a2(42); // ok (unchanged, f is a friend)
    C c2(42); // was ill-formed (implicit C constructor can't access A constructor), now ok
}
C c3(123); // still ill-formed, no access to A(int) constructor

```

This access behavior differs from that of a normal member *using-declaration*, but matches the existing design of inheriting constructors.

Proposed wording

Remove 12.9 class.inhctor, "Inheriting constructors".

Change in 3.9 basic.types paragraph 10:

A type is a *literal type* if it is:

- ...
- a class type (Clause 9) that has all of the following properties:
 - ...
 - it is an aggregate type (8.5.1) or has **or inherits (7.3.3)** at least one constexpr constructor or constructor template that is not a copy or move constructor, and
 - ...

Change in 7.1.5 dcl.constexpr paragraph 5:

For a ~~non-template, non-defaulted~~ constexpr function or ~~a non-template, non-defaulted, non-inheriting~~ constexpr

constructor **that is neither defaulted nor a template**, if no argument values exist such that ...

Change in 7.3.3 namespace.udecl paragraph 1:

A *using-declaration* introduces a **nameset of declarations** into the declarative region in which the *using-declaration* appears.

using-declaration:
using typename_{opt} nested-name-specifier unqualified-id ;

The set of declarations introduced by the *using-declaration* is found by performing qualified name lookup (3.4.3, 10.2) for the name in the *using-declaration*, excluding functions that are hidden as described below. If the *using-declaration* does not name a constructor, the *unqualified-id* The member name specified in a *using-declaration* is declared in the declarative region in which the *using-declaration* appears. [Note: Only the specified name is so declared; specifying an enumeration name in a *using-declaration* does not declare its enumerators in the *using-declaration*'s declarative region. — end note] If a *using-declaration* names a constructor (3.4.3.1), it implicitly declares a set of constructors in the class in which the *using-declaration* appears (12.9); otherwise the name specified in a *using-declaration* is as a synonym for a the set of declarations introduced by the *using-declaration* in another namespace or class. [Note: Only the specified name is so declared; specifying an enumeration name in a *using-declaration* does not declare its enumerators in the *using-declaration*'s declarative region. — end note] If the *using-declaration* names a constructor, it declares that the class inherits the set of constructor declarations introduced by the *using-declaration* from the nominated base class.

Change in 7.3.3 namespace.udecl paragraph 3:

In a *using-declaration* used as a *member-declaration*, the *nested-name-specifier* shall name a base class of the class being defined. If such a *using-declaration* names a constructor, the *nested-name-specifier* shall name a direct base class of the class being defined; otherwise it introduces the set of declarations found by member name lookup (10.2, 3.4.3.1). [Example]

Change in 7.3.3 namespace.udecl paragraph 4:

[Note] If **an a constructor or** assignment operator brought from a base class into a derived class **scope** has the signature of a copy/move **constructor or** assignment operator for the derived class (12.8), the *using-declaration* does not by itself suppress the implicit declaration of the derived class **assignment operator member**; the **copy/move assignment operator member** from the base class is hidden or overridden by the implicitly-declared copy/move **constructor or** assignment operator of the derived class, as described below.

Change in 7.3.3 namespace.udecl paragraph 8:

A *using-declaration* **for that names** a class member shall be a *member-declaration*. [Example]

Change in 7.3.3 namespace.udecl paragraph 11:

[Note: For a *using-declaration* that names a namespace, members added to the namespace after the *using-declaration* are not **in the set of introduced declarations, so they are not** considered when a use of the name is made. **[Note:** Thus, additional overloads ...

Change in 7.3.3 namespace.udecl paragraph 15:

When a *using-declaration* brings **names declarations** from a base class into a derived class **scope**, member functions and member function templates in the derived class override and/or hide member functions and member function templates with the same name, parameter-type-list (8.3.5), cv-qualification, and *ref-qualifier* (if any) in a base class (rather than conflicting). **Such hidden or overridden declarations are excluded from the set of declarations introduced by the *using-declaration*.** [Note: For using declarations that name a constructor, see 12.9. — end note] [Example:

```
struct B {  
...  
}  
  
struct B1 {  
    B1(int);  
};  
  
struct B2 {  
    B2(int);  
};  
  
struct D1 : B1, B2 {
```

```

using B1::B1;
using B2::B2;
};
D1 d1(0);    // ill-formed: ambiguous

struct D2 : B1, B2 {
    using B1::B1;
    using B2::B2;
    D2(int);    // OK: D2::D2(int) hides B1::B1(int) and B2::B2(int)
};
D2 d2(0);    // calls D2::D2(int)
]

```

Change in 7.3.3 namespace.udecl paragraph 16:

For the purpose of overload resolution, the functions **which** are introduced by a *using-declaration* into a derived class **will be** treated as though they were members of the derived class. In particular, the implicit *this* parameter shall be treated as if it were a pointer to the derived class rather than to the base class. This has no effect on the type of the function, and in all other respects the function remains a member of the base class. **Likewise, constructors that are introduced by a *using-declaration* are treated as though they were constructors of the derived class when looking up the constructors of the derived class (3.4.3.1) or forming a set of overload candidates (13.3.1.3, 13.3.1.4, 13.3.1.7). If such a constructor is selected to perform the initialization of an object of class type, all subobjects other than the base class from which the constructor originated are implicitly initialized ([class.inhctor.init]).**

Change in 7.3.3 namespace.udecl paragraph 17:

In a *using-declaration* that does not name a constructor, all members of the set of introduced declarations The access rules for inheriting constructors are specified in 12.9; otherwise all instances of the name mentioned in a *using-declaration* shall be accessible. **In a *using-declaration* that names a constructor, no access check is performed. ...**

Change in 7.3.3 namespace.udecl paragraph 18:

The alias A **synonym** created by **the** *using-declaration* has the usual accessibility for a *member-declaration*. [Note: A *using-declaration* that names a constructor does not create **aliases**; see 12.9 for the pertinent accessibility rules. **end note**] **a synonym; instead, the additional constructors are accessible if they would be accessible when used to construct an object of the corresponding base class, and the accessibility of the *using-declaration* is ignored.**
[Example]

Change in 9.2 class.mem paragraph 2:

A class is considered a completely-defined object type (3.9) (or complete type) at the closing *}* of the *class-specifier*. Within the class *member-specification*, the class is regarded as complete within function bodies, default arguments, ***using-declarations* introducing inheriting constructors (12.9)**, *exception-specifications*, and *brace-or-equal-initializers* for non-static data members (including such things in nested classes). Otherwise it is regarded as incomplete within its own class *member-specification*.

Drafting note: as observed in core issue 1487, it is not permitted to derive from an incomplete class, and the class is not regarded as complete within base-clauses, so this provision doesn't seem useful.

Change in 10 class.derived paragraph 2:

... Unless redeclared in the derived class, members of a base class are also considered to be members of the derived class. **The** **Members of a** base class **members other than constructors** are said to be *inherited* by the derived class. **Constructors of a base class can also be inherited as described in 7.3.3.** Inherited members can be referred to in expressions in the same manner as other members of the derived class, unless their names are hidden or ambiguous (10.2). ...

Add a new subclause after 12.6.2 class.base.init:

12.6.3 Initialization by inherited constructor [class.inhctor.init]

1. When a constructor for type *B* is invoked to initialize an object of a different type *D* (that is, when the constructor was inherited (7.3.3)), initialization proceeds as if a defaulted default constructor is used to initialize the *D* object and each base class subobject from which the constructor was inherited, except that the *B* subobject is initialized by the invocation of the inherited constructor. The complete initialization is considered to be a single function call; in particular, the initialization of the inherited constructor's parameters is sequenced before the

initialization of any part of the `D` object. [Example:

```
struct B1 {
    B1(int, ...) { }
};

struct B2 {
    B2(double) { }
};

int get();

struct D1 : B1 {
    using B1::B1; // inherits B1(int, ...)
    int x;
    int y = get();
};

void test() {
    D1 d(2, 3, 4); // OK: B1 is initialized by calling B1(2, 3, 4),
                  // then d.x is default-initialized (no initialization is performed),
                  // then d.y is initialized by calling get()
    D1 e;          // error: D1 has no default constructor
}

struct D2 : B2 {
    using B2::B2;
    B1 b;
};

D2 f(1.0);        // error: B1 has no default constructor

struct W { W(int); };
struct X : virtual W { using W::W; X() = delete; };
struct Y : X { using X::X; };
struct Z : Y, virtual W { using Y::Y; };
Z z(0); // OK: initialization of Y does not invoke default constructor of X

template<class T> struct Log : T {
    using T::T; // inherits all constructors from class T
    ~Log() { std::clog << "Destroying wrapper" << std::endl; }
};
```

Class template `Log` wraps any class and forwards all of its constructors, while writing a message to the standard log whenever an object of class `Log` is destroyed.]

2. If the constructor was inherited from multiple base class subobjects of type `X`, the program is ill-formed. [Example:

```
struct A { A(int); };
struct B : A { using A::A; };

struct C1 : B { using B::B; };
struct C2 : B { using B::B; };

struct D1 : C1, C2 {
    using C1::C1;
    using C2::C2;
};

struct V1 : virtual B { using B::B; };
struct V2 : virtual B { using B::B; };

struct D2 : V1, V2 {
    using V1::V1;
    using V2::V2;
};

D1 d1(0); // ill-formed: ambiguous
D2 d2(0); // OK: initializes virtual B base class, which initializes the A base class
          // then initializes the V1 and V2 base classes as if by a defaulted default constructor

struct M { M(); M(int); };
struct N : M { using M::M; };
struct O : M {};
struct P : N, O {};
P p(0); // OK: use M(0) to initialize N's base class,
        // use M() to initialize O's base class
```

]

Drafting note: This is intended to mirror the behavior when invoking a non-static member function inherited from multiple base classes.

3. When an object is initialized by an inheriting constructor, the principal constructor (12.6.2, 15.2) for the object is considered to have completed execution when the initialization of all subobjects is complete.

Do not change 15.2 except.ctor paragraph 3:

For an object of class type of any storage duration whose initialization or destruction is terminated by an exception, the destructor is invoked for each of the object's fully constructed subobjects, that is, for each subobject for which the principal constructor (12.6.2) has completed execution and the destructor has not yet begun execution, except that in the case of destruction, the variant members of a union-like class are not destroyed. ...

Change in 15.2 except.ctor paragraph 4:

Similarly, if the ~~non-delegating~~**principal** constructor for an object has completed execution and a delegating constructor for that object exits with an exception, the object's destructor is invoked. Such destruction is sequenced before entering a handler of the function-try-block of a delegating constructor for that object, if any.

Change in 15.4 except.spec paragraph 15:

The *set of potential exceptions of an expression* e is empty if e is a core constant expression (5.20). Otherwise, it is the union of the sets of potential exceptions of the immediate subexpressions of e , including default argument expressions used in a function call, combined with a set S defined by the form of e , as follows:

- If e is a function call, ...
- If e implicitly invokes a function ...
- **If e initializes an object of type D using an inherited constructor for a class of type B ([`class.inhctor.init`]), S also contains the sets of potential exceptions of the implied constructor invocations for subobjects of D that are not subobjects of B (including default argument expressions used in such invocations) as selected by overload resolution, and the sets of potential exceptions of the initialization of non-static data members from *brace-or-equal-initializers* (12.6.2).**
- ...

Change in 15.4 except.spec paragraph 16:

Given an **implicitly-declared special** member function f of some class X , ~~where f is an inheriting constructor (12.9) or an implicitly-declared special member function,~~ the *set of potential exceptions of the implicitly-declared **special** member function* f consists of all the members from the following sets: ...

Change in 15.4 except.spec paragraph 17:

An ~~inheriting constructor (12.9) and an~~ implicitly-declared special member function (Clause 12) **are** considered to have an implicit exception specification, as follows, where S is the set of potential exceptions of the implicitly-declared **special** member function.

- ...

[Note: An instantiation of an inheriting constructor template has an implied exception specification as if it were a non-template inheriting constructor. — end note] [Example: ...]